30-Day FreeRTOS Course for ESP32 Using ESP-IDF

(Day 24)

“Power Management with FreeRTOS on ESP32”

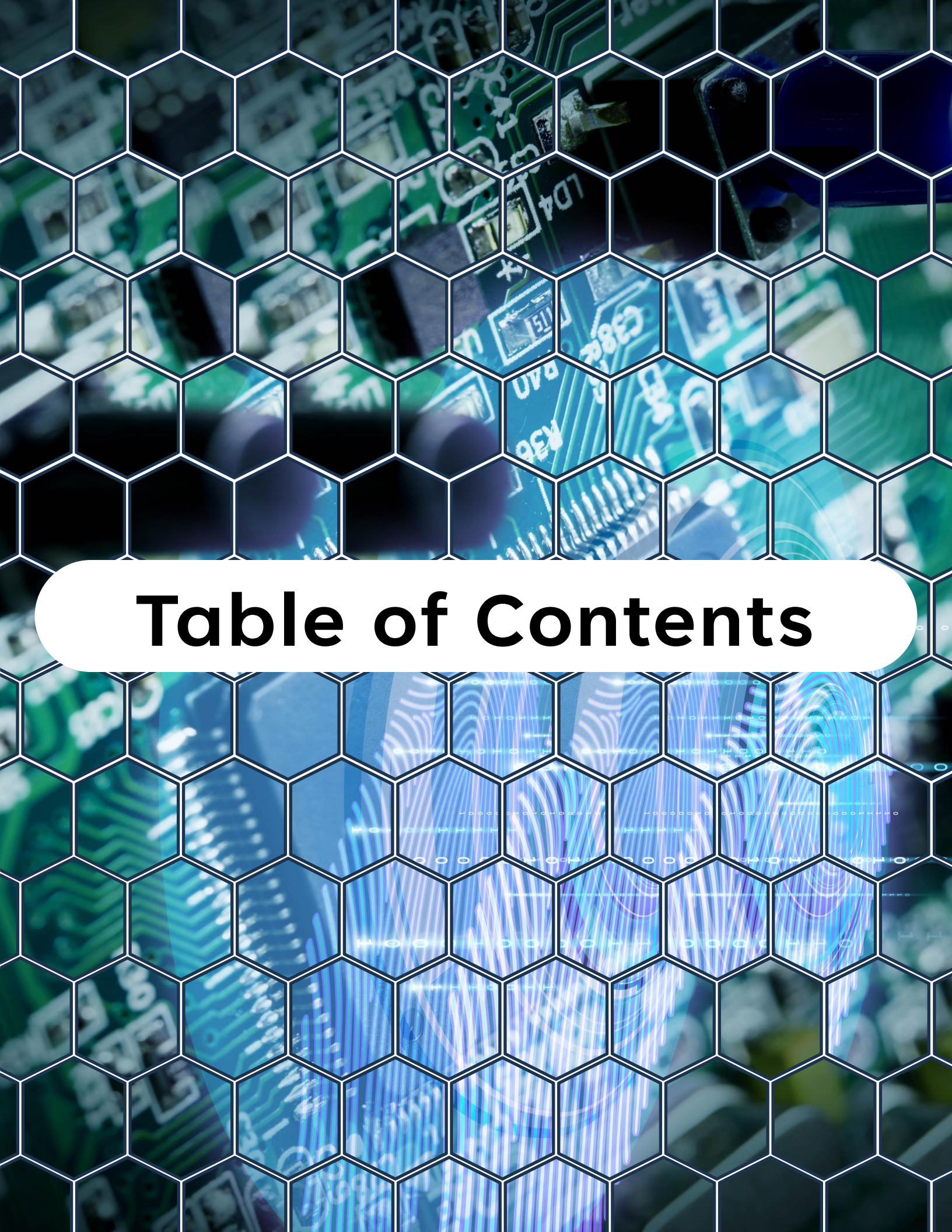
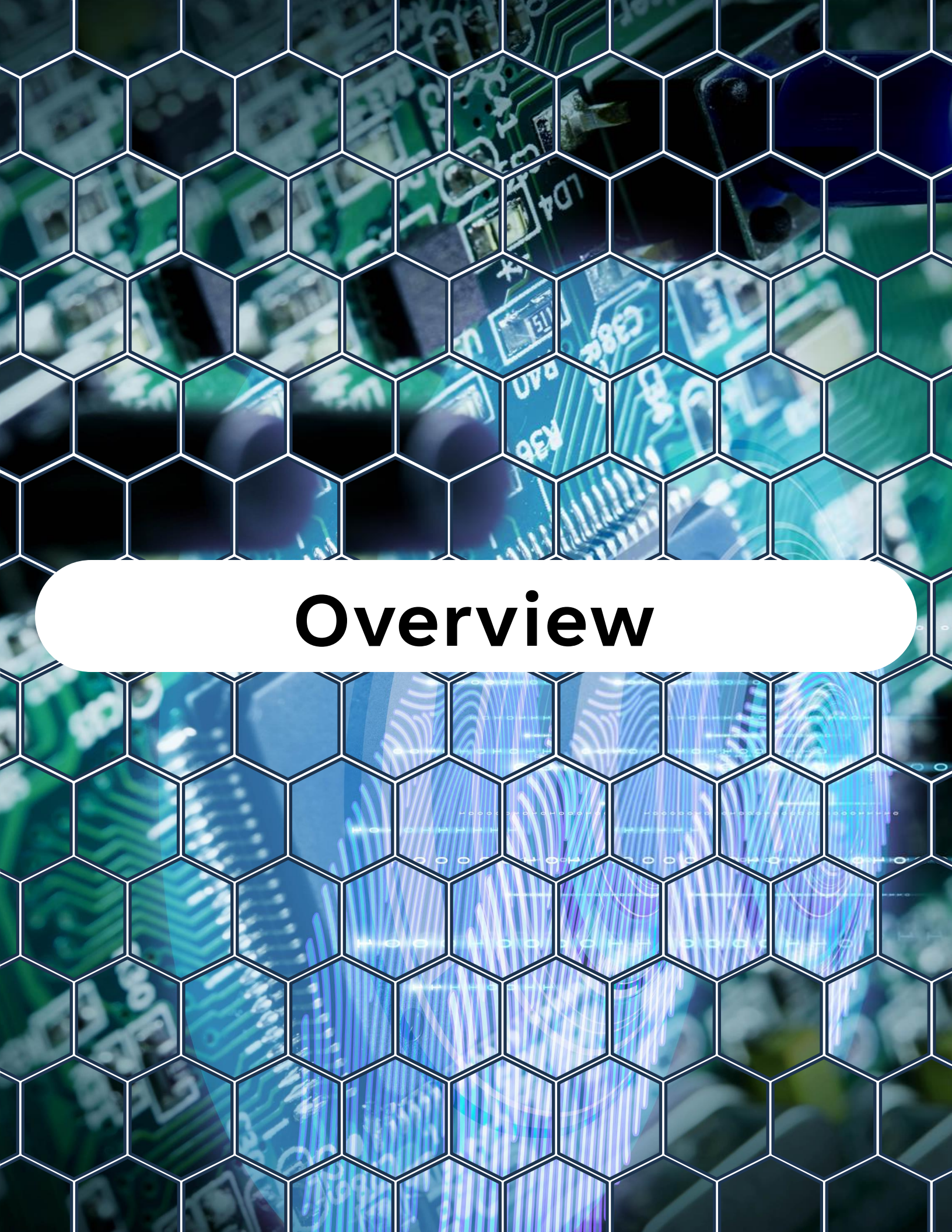


Table of Contents

Table of Contents



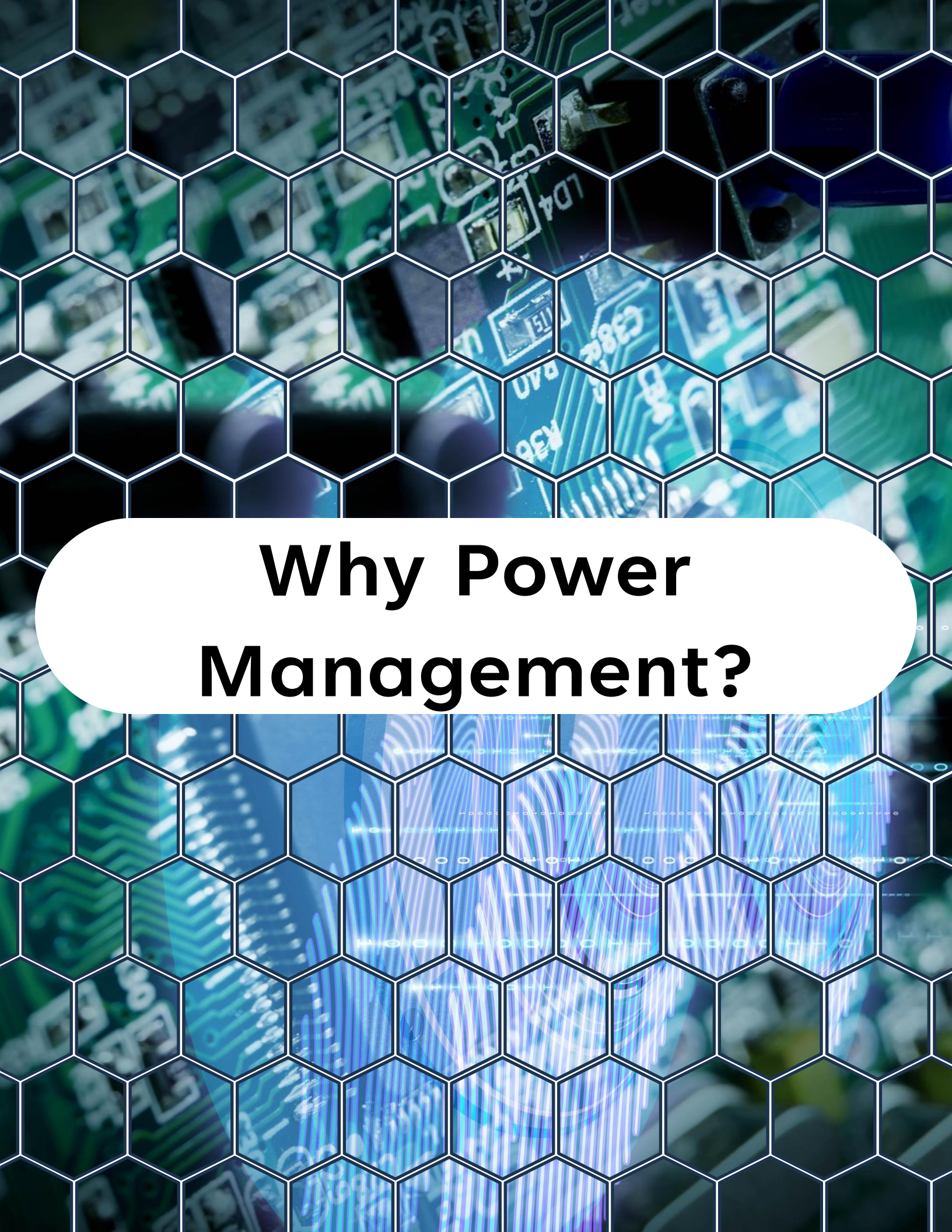
Overview

Overview

On **Day 24**, you'll learn:

- Why power management matters in ESP32 FreeRTOS applications
- The power modes of ESP32 (active, light sleep, deep sleep)
- How FreeRTOS features like tickless idle interact with ESP-IDF's power management
- APIs for entering sleep modes and waking up
- A practical example: combining FreeRTOS tasks with ESP-IDF sleep APIs
- Best practices for designing low-power FreeRTOS applications

By the end of this lesson, you'll know how to reduce power consumption in ESP32 systems while keeping FreeRTOS tasks responsive.



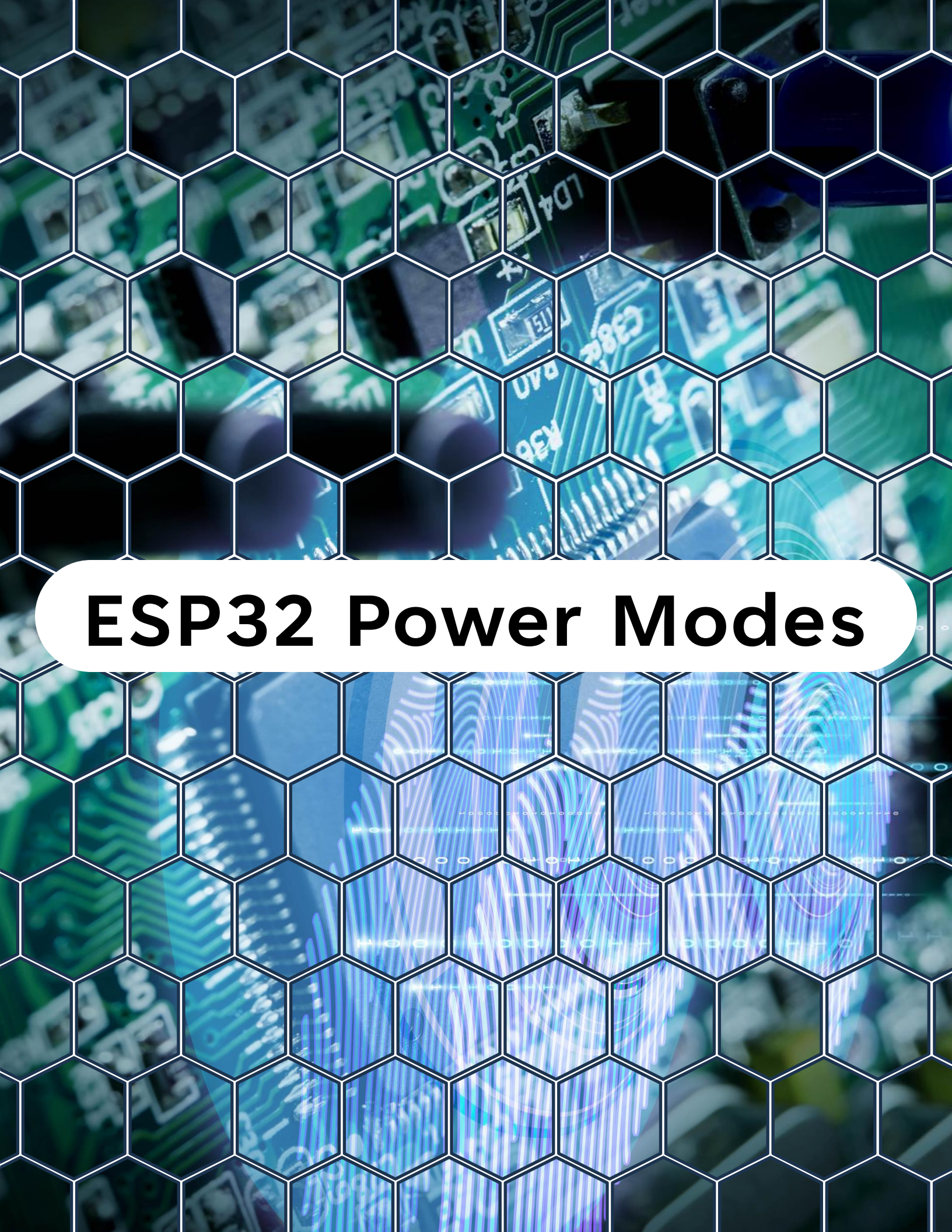
Why Power Management?

Why Power Management?

ESP32 is often used in battery-powered IoT devices. Reducing consumption is critical for:

- Extending battery life
- Meeting thermal and efficiency requirements
- Enabling long-term deployments in remote environments

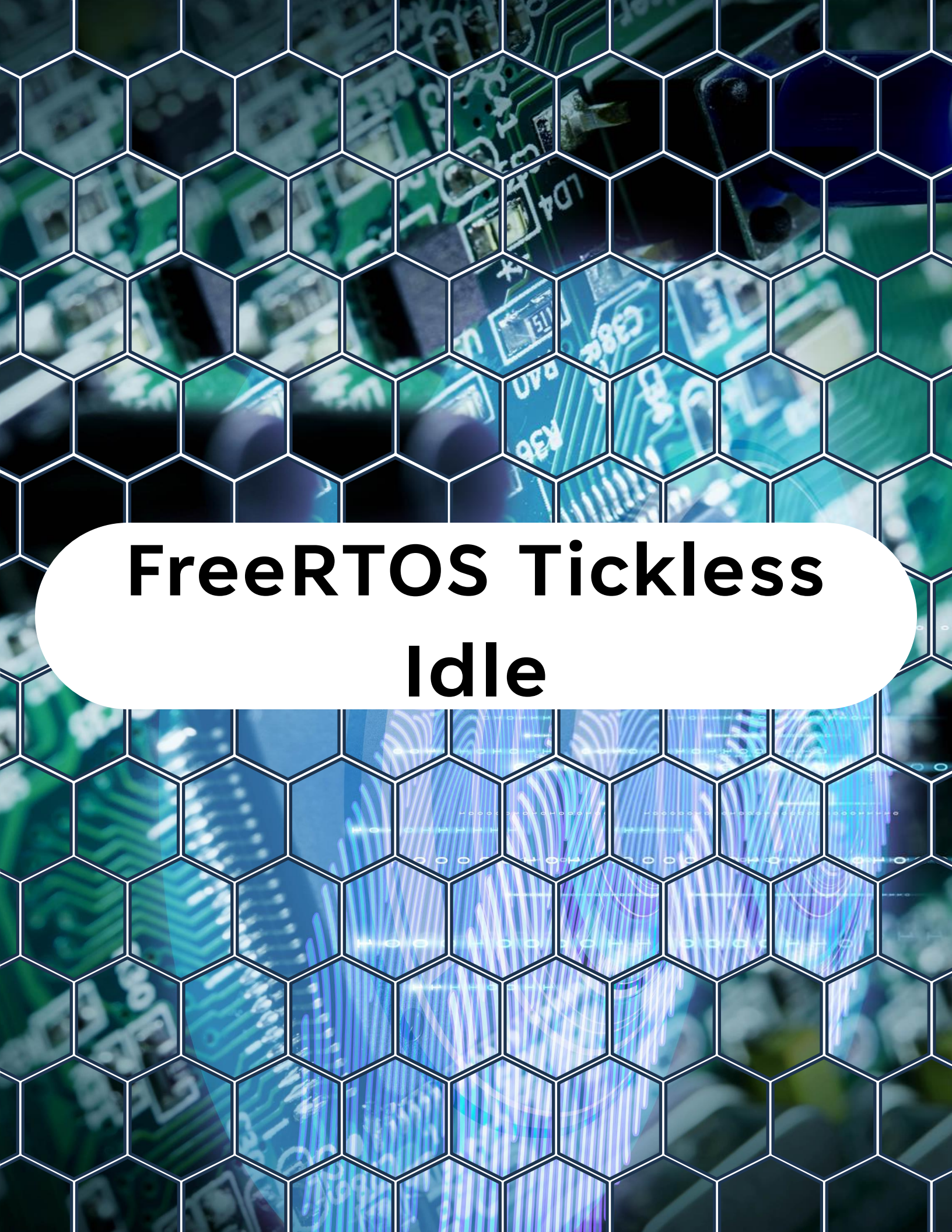
📌 Power savings are achieved by controlling CPU idle time and using sleep modes.


ESP32 Power Modes

ESP32 Power Modes

ESP-IDF provides several power modes:

Mode	Description
Active	CPU and peripherals fully powered.
Light Sleep	Most peripherals off, RAM retained, wakeup via timer/interrupt.
Deep Sleep	CPU + most peripherals off, only RTC domain active, wakeup via RTC timer, GPIO, or ULP.



FreeRTOS Tickless Idle

FreeRTOS Tickless Idle

- When **tickless idle** is enabled (Day 22), FreeRTOS suppresses unnecessary ticks when the system is idle.
- ESP-IDF integrates this with **light sleep mode**:
  - CPU sleeps until the next task wakeup time.
  - Saves significant energy when tasks are idle for long periods.

Enable in menuconfig:

Component config → FreeRTOS → Tickless idle support

APIs for Power Management in ESP-IDF

APIs for Power Management in ESP-IDF

- **Light Sleep**

```c
#include "esp_sleep.h"

// Enter light sleep mode
esp_light_sleep_start();
```

CPU stops, memory retained, peripherals paused.

- **Deep Sleep**

```c
#include "esp_sleep.h"

// Put the device into deep sleep for a
// specified time in microseconds
esp_deep_sleep(time_in_us);
```

Chip resets after wakeup, execution restarts from app_main.

APIs for Power Management in ESP-IDF

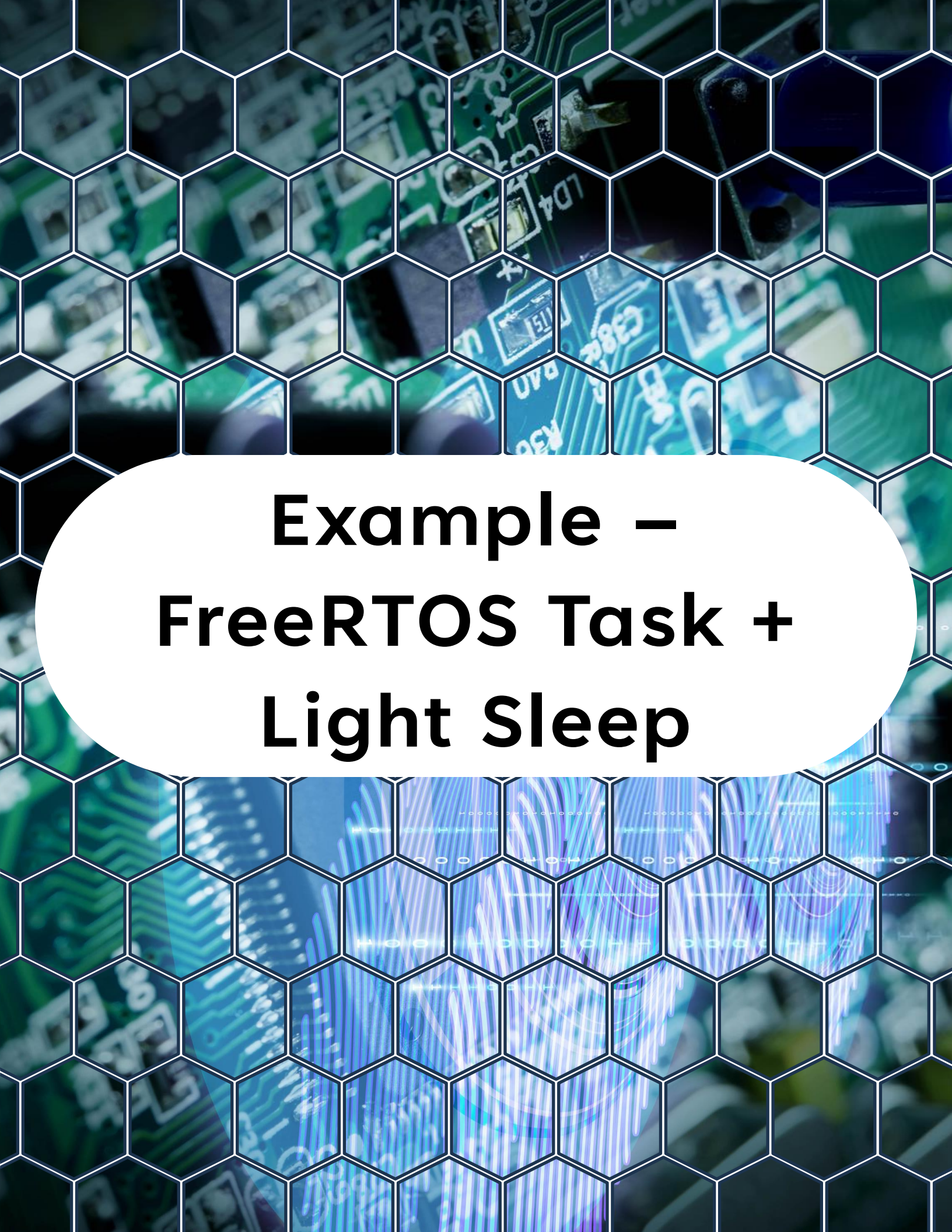
- **Wakeup Configuration**

```c
#include "esp_sleep.h"

// Set timer wakeup
esp_sleep_enable_timer_wakeup(us);

// Enable wakeup from GPIO
esp_sleep_enable_ext0_wakeup(gpio, level);

// Enable wakeup from multiple GPIOs
esp_sleep_enable_ext1_wakeup(mask, mode);
```



Example – FreeRTOS Task + Light Sleep

Example – FreeRTOS Task + Light Sleep

Code Example

```c
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_sleep.h"
#include "esp_log.h"

#define TAG "DAY24"

// Simulated sensor reading task
void sensor_task(void *pvParameter) {
    while (1) {
        ESP_LOGI(TAG, "Reading sensor...");
        vTaskDelay(pdMS_TO_TICKS(2000)); // Simulate work
    }
}

// Main application entry point
void app_main() {
    // Enable timer wakeup every 5 seconds
    esp_sleep_enable_timer_wakeup(5000000ULL);

    // Create a sensor task
    xTaskCreate(sensor_task, "SensorTask", 2048, NULL, 5, NULL);

    while (1) {
        ESP_LOGI(TAG, "System entering light sleep for 5s");

        // Enter light sleep
        esp_light_sleep_start();

        ESP_LOGI(TAG, "Woke up from light sleep");
```

Example – FreeRTOS Task + Light Sleep

```
    while (1) {
        ESP_LOGI(TAG, "System entering light sleep for 5s");

        // Enter light sleep
        esp_light_sleep_start();

        ESP_LOGI(TAG, "Woke up from light sleep");
    }
}
```

Expected Behavior

- The system enters **light sleep** every 5 seconds.
- Wakes up due to the **timer wakeup source**.
- Continues running FreeRTOS tasks after waking.

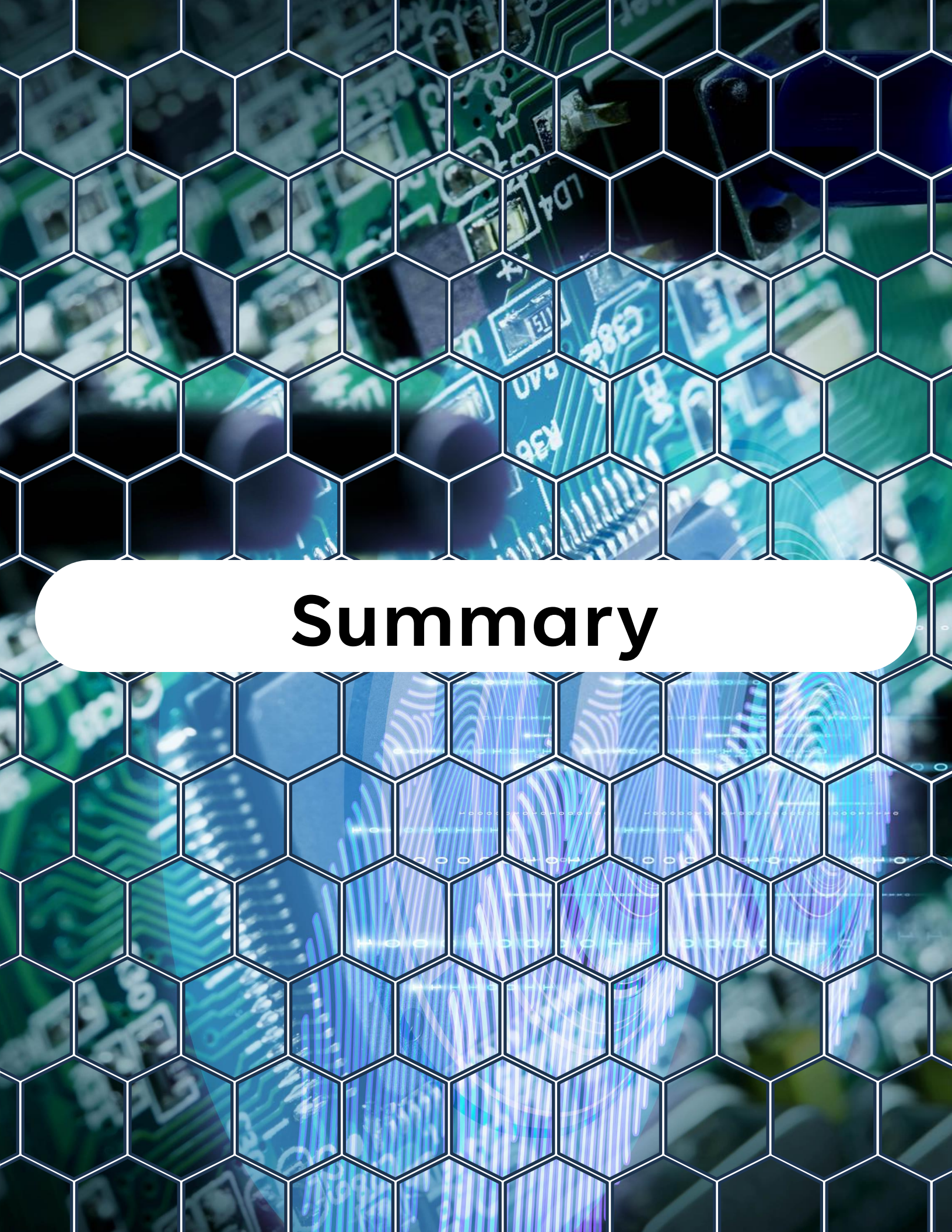
Serial output:

```
I (1000) DAY24: Reading sensor...
I (2000) DAY24: System entering light sleep for 5s
I (7000) DAY24: Woke up from light sleep
I (7000) DAY24: Reading sensor...
```

Best Practices

Best Practices

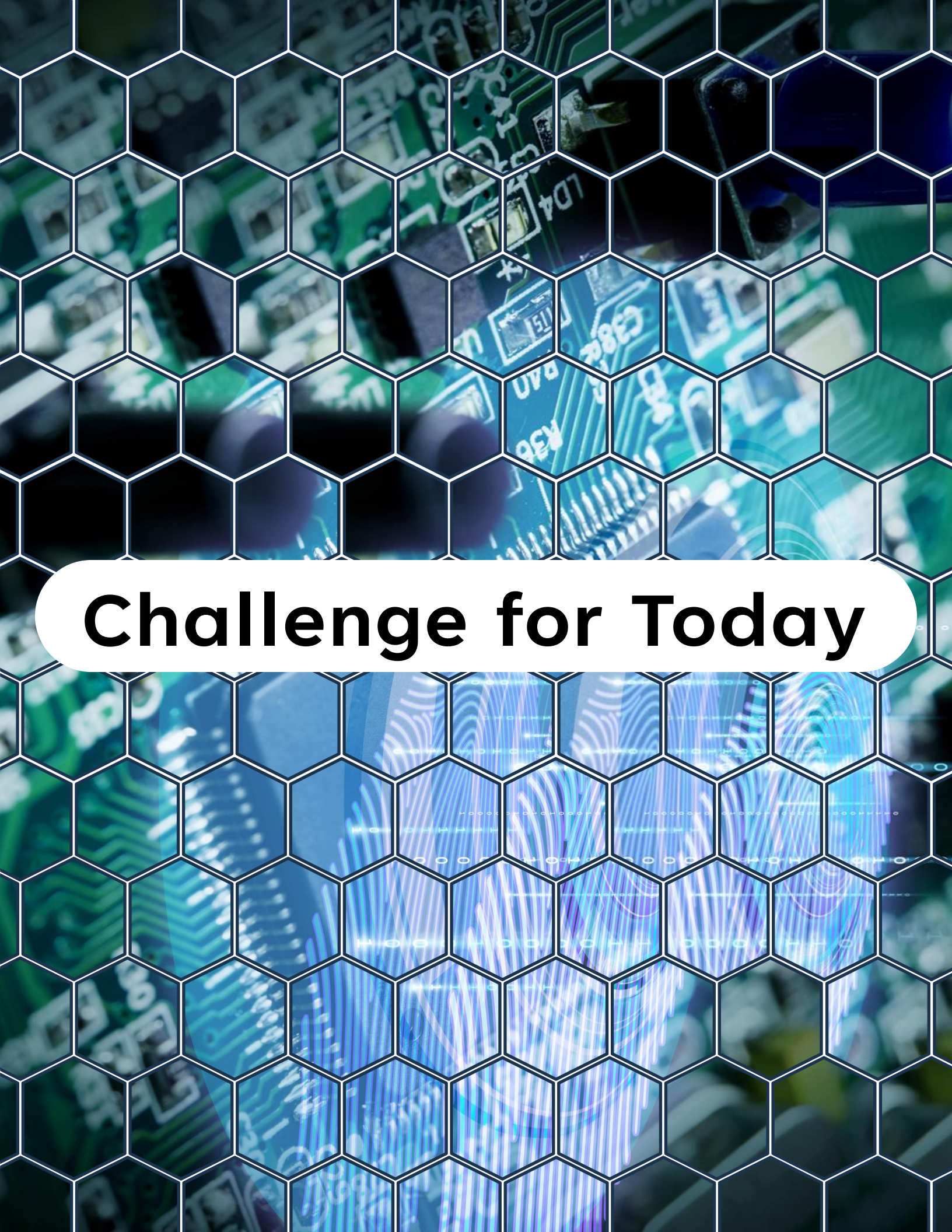
- ✅ Use **tickless idle** to let ESP-IDF manage idle CPU sleep automatically.
- ✅ Use **light sleep** for short idle times; **deep sleep** for long power-off periods.
- ✅ Keep ISR wakeup sources minimal to avoid unnecessary wakeups.
- ✅ Save state in **NVS** or **RTC memory** before deep sleep.
- ✅ Profile power consumption with tools like **ESP-IDF power monitor**.



Summary

Summary

- ESP32 supports **active, light sleep, and deep sleep modes**.
- FreeRTOS **tickless idle** helps save power during idle time.
- ESP-IDF provides APIs for **light sleep and deep sleep** with configurable wakeup sources.
- Combining FreeRTOS tasks with ESP-IDF power management enables efficient **low-power designs**.


Challenge for Today

Challenge for Today

🛠️ Extend today's project by:

1. Configure ESP32 to enter **deep sleep** for 10 seconds.
2. Use `esp_sleep_enable_ext0_wakeup(GPIO_NUM_0, 0)` to wake on button press.
3. On wakeup, print whether the system woke due to **timer** or **GPIO**.
4. Compare current draw between **light sleep** and **deep sleep** modes.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

https://www.linkedin.com/in/yamil-garcia
https://www.youtube.com/@LearningByTutorials
https://github.com/god233012yamil